Reinforcement Learning for Text Flappy Bird: Monte Carlo Control vs Sarsa(λ)

Mayara AYAT

CentraleSupélec

1 Introduction

We study reinforcement learning in the Text Flappy Bird environment, where the objective is to maximize survival time using a discrete state representation and two actions: *do_nothing* and *flap*. We compare First-visit Monte Carlo control vs Sarsa(λ) (Sutton & Barto, Section 12.7). The comparison focuses on performance, value functions, parameter sensitivity, and generalization.

2 Experimental Setup

2.1 Environment

We use `TextFlappyBird-v0`, a simplified Flappy Bird environment with a discrete state (x, y) describing the relative position to the next obstacle and two actions. This representation enables tabular learning and direct visualization of $Q(s, a)$ and $V(s)$.

2.2 Text vs Screen-Based Environments

Two versions exist:

- **Text-based**: low-dimensional, discrete, interpretable
- **Screen-based**: high-dimensional visual observations

The text-based version is computationally efficient and suitable for tabular methods but simplifies the problem. The screen-based version is more realistic but requires function approximation and is significantly harder to train.

2.3 Applicability to Screen-Based Environment

The tabular agents used in this work cannot be directly applied to the original version of the environment due to the high-dimensional visual input. In this setting, the state space becomes continuous and requires function approximation (e.g., neural networks). While Monte Carlo and Sarsa(λ) remain conceptually valid, they must be extended to deep reinforcement learning methods.

2.4 Training and Evaluation

Both agents are trained for 5000 episodes with ϵ-greedy exploration (decayed for Sarsa(λ)). Evaluation is performed over 100 episodes using the greedy policy. Performance is measured by mean return and standard deviation.

3 Methods

3.1 Monte Carlo Control

We use first-visit on-policy Monte Carlo with ϵ-greedy exploration. The update is: $Q(s, a) \leftarrow Q(s, a) + \alpha\big(G_t - Q(s, a)\big), \quad \alpha = \frac{1}{N(s,a)}$.

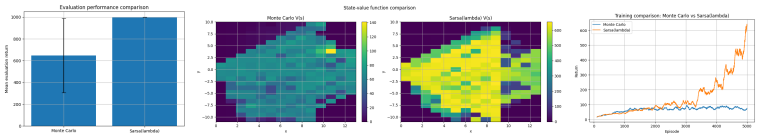

Fig. 1. Left: Evaluation performance (mean return ± std). Center: state-value functions $V(s)$ (Monte Carlo vs Sarsa(λ)). Right: training curves (average return).

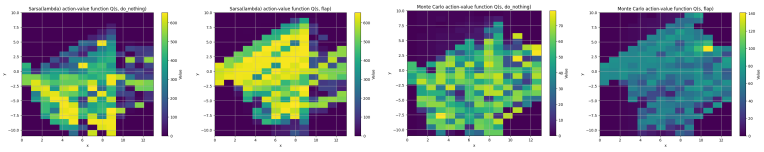

Fig. 2. Action-value functions $Q(s, a)$: Sarsa(λ) (two on the left) and Monte Carlo (two on the right); *do_nothing* (left), *flap* (right).

3.2 Sarsa(λ)

We implement tabular Sarsa(λ) with accumulating eligibility traces. The TD error is: $\delta_t = R_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)$. Traces are updated as: $E(s, a) \leftarrow \gamma\lambda E(s, a), \quad E(s_t, a_t) \leftarrow E(s_t, a_t) + 1$.

The value update is: $Q(s, a) \leftarrow Q(s, a) + \alpha\delta_t E(s, a)$.

4 Results

4.1 Performance Comparison

The evaluation performance is:

- Monte Carlo: 646.08 ± 340.70
- Sarsa(λ): 1000 ± 0

As shown in Fig. 1, Sarsa(λ) consistently achieves the maximum return with zero variance, indicating convergence to a stable and near-optimal policy. In contrast, Monte Carlo exhibits significantly lower performance and very high variance, reflecting unstable learning due to reliance on full-episode returns.

4.2 Learning Dynamics

Fig. 1 highlights a clear difference in convergence behavior. Sarsa(λ) converges significantly faster than Monte Carlo, reaching high returns after fewer episodes, while Monte Carlo quickly plateaus at a much lower performance level. Moreover, Monte Carlo exhibits persistent fluctuations, whereas Sarsa(λ) shows a more structured learning trajectory. This demonstrates the benefit of bootstrapping and eligibility traces for faster and more stable convergence.

4.3 State-Value Functions

Fig. 1 shows that both methods identify a high-value corridor corresponding to safe states. However, Monte Carlo produces noisy and irregular estimates, with local inconsistencies due to high-variance returns. In contrast, Sarsa(λ) yields smoother and more coherent value functions, capturing the global structure of the environment more effectively. This reflects improved credit assignment through temporal-difference updates.

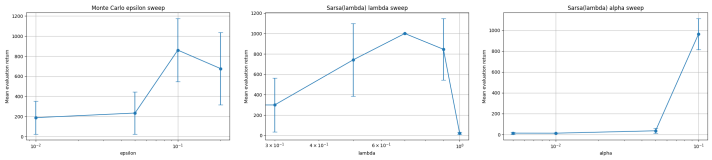

Fig. 3. Parameter sweeps: ϵ (Monte Carlo), λ and α (Sarsa(λ)).

4.4 Action-Value Functions

Fig. 2 further emphasizes the difference in learning quality. Sarsa(λ) produces well-structured and high-magnitude action-values, clearly separating regions where each action is optimal. In contrast, Monte Carlo estimates are noisier and less discriminative, leading to less reliable action selection. This directly explains the performance gap observed in Fig. 1.

4.5 Parameter Sensitivity

Fig. 3 shows that Monte Carlo is moderately sensitive to ϵ. Performance is maximized for intermediate values, which balance exploration and exploitation. Too small ϵ leads to insufficient exploration, while too large ϵ introduces excessive randomness.

Sarsa(λ) is significantly more sensitive to its parameters. Performance peaks around $\lambda \approx 0.7$, where eligibility traces effectively propagate rewards without introducing excessive variance. For $\lambda = 1$, performance degrades due to Monte Carlo-like behavior. Similarly, a sufficiently large learning rate α is required for efficient learning, while too small values slow convergence.

4.6 Generalization Across Configurations

Hard setting (gap = 2):

- Monte Carlo: 25.56 ± 17.53
- Sarsa(λ): 26.91 ± 17.24

Both methods perform poorly, with low returns and high variance. This indicates that the learned policies do not generalize to more difficult environments and are highly specialized to the training configuration.

Easy setting (gap = 5):

- Monte Carlo: 956.78 ± 173.24
- Sarsa(λ): 1000 ± 0

Both methods perform well in easier settings, but Sarsa(λ) remains perfectly stable while Monte Carlo still exhibits variability. This reveals an asymmetry: both agents transfer to easier environments but fail in harder ones.

4.7 Discussion

The results reveal a trade-off between variance and learning efficiency. Monte Carlo relies on full-episode returns, resulting in high variance, noisy value estimates, and unstable performance. In contrast, Sarsa(λ) uses bootstrapping and eligibility traces to achieve faster reward propagation, smoother value functions, and stable convergence. This improved performance comes at the cost of greater sensitivity to hyperparameters. In particular, λ controls the bias–variance trade-off, while α governs convergence speed. Overall, Sarsa(λ) outperforms Monte Carlo in performance, stability, and representation quality, but requires careful tuning.